University of Wah

Department of Computer Science

BS Cyber Security

SECURE SOFTWARE DESIGN AND DEVELOPMENT

Project Report

Title: Secure Software Deployment Using ModSecurity and OWASP Core Rule Set

Submitted to: Ma'am Muniba Khan

Group Members:

Hassan Iftikhar (UW-23-CY-BS-002)

Muhammad Azfar Waqas (UW-23-CY-BS-013)

Ibrar Ul Hassan Shami (UW-23-CY-BS-018)

Section: BS-CYS-5N

Table of Contents

1. Introduction

Web applications are one of the most common targets of cyber-attacks such as SQL Injection and Cross-Site Scripting (XSS). These attacks exploit weaknesses in application input handling and can lead to unauthorized access, data leakage, or complete system compromise.

This project focuses on applying **Secure Software Design (SSD)** principles by designing a **secure deployment architecture** for a vulnerable web application. Instead of modifying the application's source code, security is enforced using a **Web Application Firewall (WAF)** implemented through **Apache, ModSecurity, and OWASP Core Rule Set (CRS)**.

1.1 Purpose

The purpose of this project is to:

- Identify common web application vulnerabilities
- Design a secure deployment architecture
- Implement a Web Application Firewall as a compensating security control
- Detect and block SQL Injection and XSS attacks in real time
- Log malicious activities for auditing and analysis
- Demonstrate SSD principles such as defense-in-depth and secure deployment

1.2 System Architecture

The system follows a layered security architecture:

Attacker (Ubuntu)

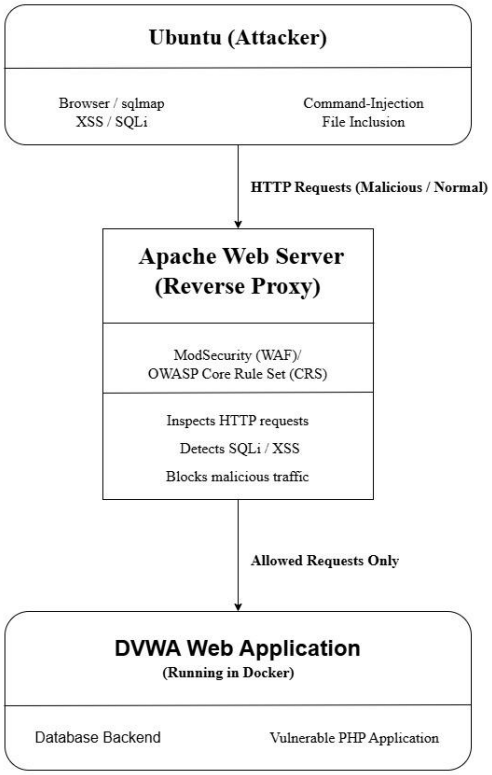
↓

Apache Reverse Proxy + ModSecurity (WAF)

↓

DVWA Web Application (Docker)

All incoming HTTP requests are inspected by the WAF. Malicious requests are blocked, while legitimate traffic is forwarded to the application.


Ubuntu (Attacker)
Browser / sqlmap
XSS / SQLi
Command-Injection
File Inclusion
HTTP Requests (Malicious / Normal)
Apache Web Server
(Reverse Proxy)
ModSecurity (WAF)/
OWASP Core Rule Set (CRS)
Inspects HTTP requests
Detects SQLi / XSS
Blocks malicious traffic
Allowed Requests Only
DVWA Web Application
(Running in Docker)
Database Backend
Vulnerable PHP Application

1.3 Targeted Operating Systems

The primary operating systems used in this project are:

- **Kali Linux:** Hosting DVWA, Apache, ModSecurity, and OWASP CRS
- **Ubuntu Linux:** Used as the attacker machine

The system is tested and optimized for Linux-based environments.

1.4 Compatibility with Other Systems

The project can be adapted for:

- Other Linux distributions
- Cloud-based environments
- Legacy web applications

Minimal changes are required, mainly in firewall rules and deployment configurations.

1.5 Targeted Hardware Platforms

The project is designed to run on:

- Personal computers (4–8 GB RAM recommended)
- Virtual machines
- Cloud instances
- Academic lab systems

Docker ensures lightweight and portable deployment.

2. Overall Description

This project demonstrates how secure software design principles can protect vulnerable applications without altering their internal code.

2.1 Manageability

The system is designed with maintainability in mind:

- Modular configuration files
- Clear separation between application and security layers
- Easy service restart after system reboot
- Logs stored centrally for monitoring

2.2 User Classes and Characteristics

Security Administrator

- High technical expertise
- Configures ModSecurity and CRS rules
- Monitors logs and blocked attacks

Developer

- Moderate to high expertise
- Understands secure deployment architecture
- Uses logs to identify security weaknesses

Academic Evaluator / Instructor

- Reviews security design and effectiveness
- Evaluates SSD principles applied

2.3 Design and Implementation Constraints

- No modification to DVWA source code
- Security applied at deployment level
- Limited to lab-based environment
- Requires Apache and Docker services

3. Threat Modeling

Threat modeling is an essential activity in **Secure Software Design (SSD)** that helps identify potential threats, attackers, vulnerable components, and security controls within a system. In this project, threat modeling is used to analyze how web-based attacks target the application and how the designed security architecture mitigates these threats.

3.1 Threat Model Overview

The primary asset in this system is the **web application (DVWA)** and its associated data. The system is exposed to common web application threats such as **SQL Injection** and **Cross-Site Scripting (XSS)**. An attacker attempts to exploit these vulnerabilities by sending malicious HTTP requests.

To mitigate these threats, a **Web Application Firewall (WAF)** using ModSecurity and OWASP Core Rule Set is placed between the attacker and the application.

3.2 Identified Assets

The following assets are protected in this system:

- Web application source and logic
- Backend database
- User input and session data
- Server resources (CPU, memory)
- Application availability

3.3 Identified Threats

Threat	**Description**
SQL Injection	Unauthorized database access via malicious input
Cross-Site Scripting (XSS)	Execution of malicious JavaScript in user browsers
Input Manipulation	Tampering with HTTP parameters
Denial of Service (Application-Level)	Excessive malicious requests

3.4 Threat Mitigation Strategy

Threat	Mitigation Mechanism
SQL Injection	OWASP CRS SQLi detection rules
XSS	Script and payload filtering
Input Manipulation	Request inspection and validation
Attack Visibility	Logging and monitoring

All mitigation controls are applied **before the request reaches the application**, ensuring secure software deployment.

3.5 SSD Relevance of Threat Modeling

This threat model demonstrates the application of Secure Software Design principles by:

- Identifying threats at the **design phase**
- Protecting software through **secure architecture**
- Applying **defense-in-depth**
- Using **compensating security controls**

Threat modeling ensures that security is not an afterthought but an integral part of the software deployment process.

4. Non-Functional Requirements

4.1 Performance Requirements

- Real-time request inspection
- Minimal latency for legitimate traffic
- Efficient handling of concurrent requests

4.2 Security Requirements

- Blocking of SQL Injection attacks
- Blocking of XSS payloads
- Logging of all malicious attempts
- Isolation of attacker and victim networks

4.3 Software Quality Attributes

Reliability

- Continuous protection even after reboot

Maintainability

- Simple configuration and restart

Scalability

- Can handle increased traffic with tuning

Usability

- Easy to demonstrate and manage

5. Other Requirements

5.1 Hardware Interface

- Network interface for HTTP traffic
- Virtualized interfaces using VirtualBox

5.2 Communication Interface

- HTTP/HTTPS communication
- Internal Docker networking
- Secure logging mechanisms

6. Execution

Steps to Run the System:

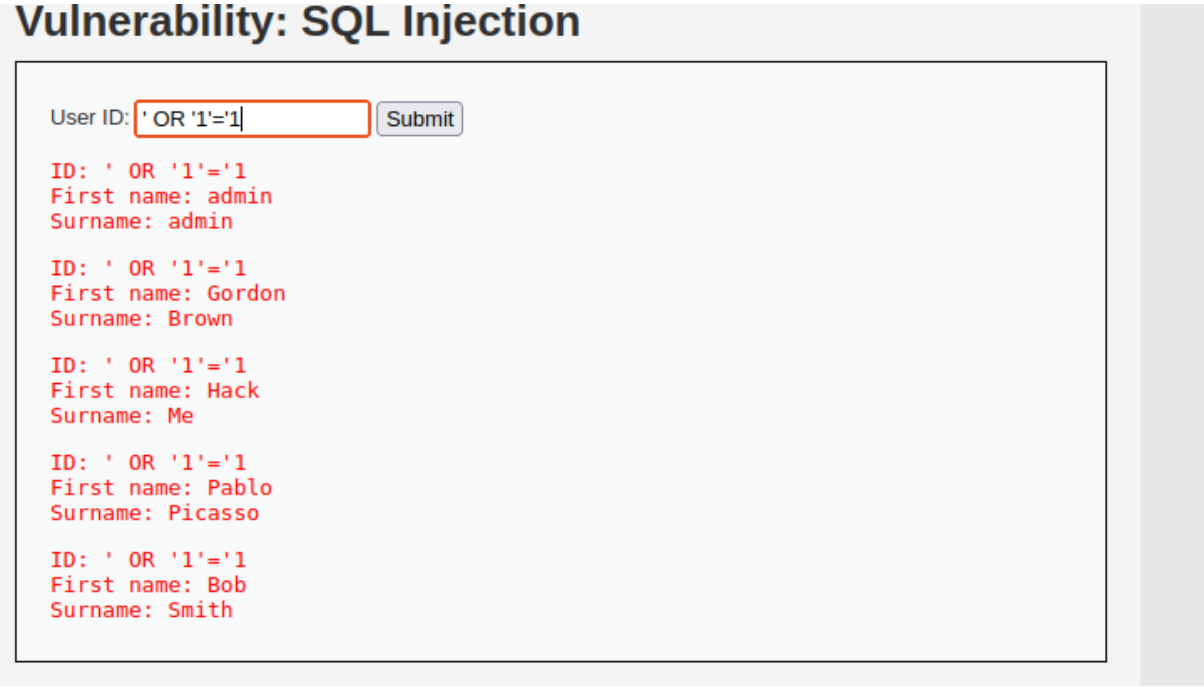
1. Start Kali and Ubuntu virtual machines
2. Start Docker and DVWA container on Kali
3. Start Apache and ModSecurity services
4. Access DVWA through WAF (Port 8081)
5. Launch SQL Injection and XSS attacks from Ubuntu
6. Observe blocked responses and logs

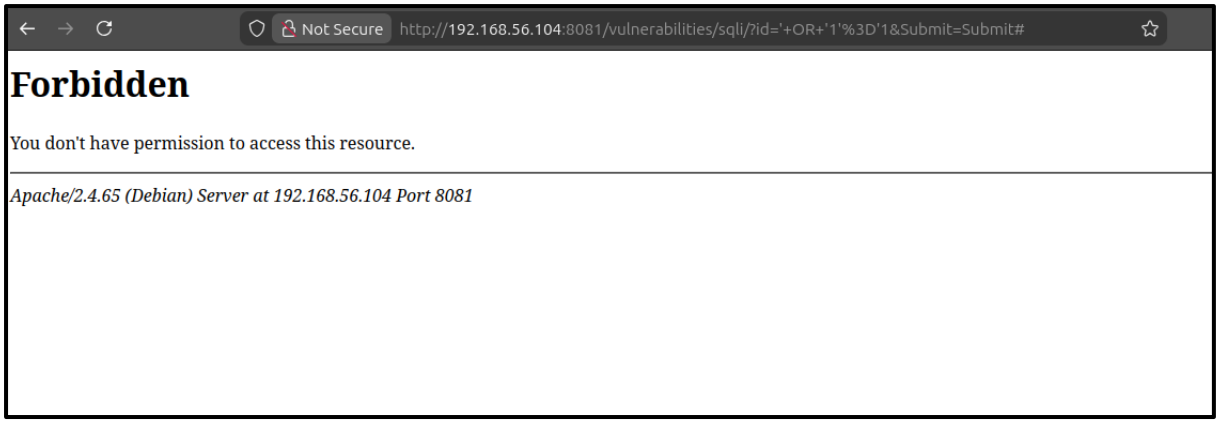
```
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0d:ce:f0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic eth0
       valid_lft 583sec preferred_lft 583sec
```

```
┌──(mak㉿mak)-[~]
└─$ sudo docker run -d -p 8080:80 vulnerables/web-dvwa

30dd51a434d7339d20ebf3dfacece91e538de9680b38dae2048a466ccef2f558

┌──(mak㉿mak)-[~]
└─$ sudo systemctl start apache2
```






```
--c5d95e78-A--
[27/Jan/2026:23:10:14.431344 --0600] aXmaNiwJUvbRokpxhgULRgAAAAE 192.168.56.103 50474 192.168.56.104 8081
--c5d95e78-B--
GET /vulnerabilities/sqli/?id=%27+OR+%271%27%3D%271&Submit=Submit HTTP/1.1
Host: 192.168.56.104:8081
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.56.104:8081/vulnerabilities/sqli/
Cookie: PHPSESSID=5ebbghdsnd6ssdl6q69assl964; security=low
Upgrade-Insecure-Requests: 1
Priority: u=0, i

--c5d95e78-F--
HTTP/1.1 403 Forbidden
Content-Length: 281
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1

--c5d95e78-E--
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.65 (Debian) Server at 192.168.56.104 Port 8081</address>
</body></html>
```

```
Message: Warning. Pattern match "(?:^([\\d.]+|\\[[\\da-f:]+\\]|[\\da-f:]+)(:[\\d]+)?$)" at REQUEST_HEADERS:Host. [file "/etc/modsecurity/coreruleset/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line "729"] [id "920350"] [msg "Host header is a numeric IP address"] [data "192.168.56.104:8081"] [severity "WARNING"] [ver "OWASP_CRS/4.22.0-dev"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-protocol"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/PROTOCOL-ENFORCEMENT"] [tag "capec/1000/210/272"]
Message: Warning. detected SQLi using libinjection with fingerprint 's&sos' [file "/etc/modsecurity/coreruleset/rules/REQUEST-942-APPLICATION-ATTACK-SQLI.conf"] [line "66"] [id "942100"] [msg "SQL Injection Attack Detected via libinjection"] [data "Matched Data: s&sos found within ARGS:id: ' OR '1'='1"] [severity "CRITICAL"] [ver "OWASP_CRS/4.22.0-dev"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-sqli"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/ATTACK-SQLI"] [tag "capec/1000/152/248/66"]
Message: Access denied with code 403 (phase 2). Operator GE matched 5 at TX:blocking_inbound_anomaly_score. [file "/etc/modsecurity/coreruleset/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "233"] [id "949110"] [msg "Inbound Anomaly Score Exceeded (Total Score: 8)"] [ver "OWASP_CRS/4.22.0-dev"] [tag "anomaly-evaluation"] [tag "OWASP_CRS"]
Message: Warning. Unconditional match in SecAction. [file "/etc/modsecurity/coreruleset/rules/RESPONSE-980-CORRELATION.conf"] [line "98"] [id "980170"] [msg "Anomaly Scores: (Inbound Scores: blocking=8, detection=8, per_pl=8-0-0-0, threshold=5) - (Outbound Scores: blocking=0, detection=0, per_pl=0-0-0-0, threshold=4) - (SQLI=5, XSS=0, RFI=0, LFI=0, RCE=0, PHPI=0, HTTP=0, SESS=0, COMBINED_SCORE=8)"] [ver "OWASP_CRS/4.22.0-dev"] [tag "reporting"] [tag "OWASP_CRS"]
Apache-Error: [file "apache2_util.c"] [line 286] [level 3] ModSecurity: Warning. Pattern match "(?:^([\\\\\\\\d.]+|\\\\\\\\[[\\\\\\\\da-f:]+\\\\\\\\]|[\\\\\\\\da-f:]+)(:[\\\\\\\\d]+)?$)" at REQUEST_HEADERS:Host. [file "/etc/modsecurity/coreruleset/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line "729"] [id "920350"] [msg "Host header is a numeric IP address"] [data "192.168.56.104:8081"] [severity "WARNING"] [ver "OWASP_CRS/4.22.0-dev"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-protocol"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/PROTOCOL-ENFORCEMENT"] [tag "capec/1000/210/272"] [hostname "192.168.56.104"] [uri "/vulnerabilities/sqli/"] [unique_id "aXmaNiwJUvbRokpxhgULRgAAAAE"]
Apache-Error: [file "apache2_util.c"] [line 286] [level 3] ModSecurity: Warning. detected SQLi using libinjection with fingerprint 's&sos' [file "/etc/modsecurity/coreruleset/rules/REQUEST-942-APPLICATION-ATTACK-SQLI.conf"] [line "66"] [id "942100"] [msg "SQL Injection Attack Detected via libinjection"] [data "Matched Data: s&sos found within ARGS:id: ' OR '1'='1"] [severity "CRITICAL"] [ver "OWASP_CRS/4.22.0-dev"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-sqli"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "OWASP_CRS/ATTACK-SQLI"] [tag "capec/1000/152/248/66"] [hostname "192.168.56.104"] [uri "/vulnerabilities/sqli/"] [unique_id "aXmaNiwJUvbRokpxhgULRgAAAAE"]
Apache-Error: [file "apache2_util.c"] [line 286] [level 3] ModSecurity: Access denied with code 403 (phase 2). Operator GE matched 5 at TX:blocking_inbound_anomaly_score. [file "/etc/modsecurity/coreruleset/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "233"] [id "949110"] [msg "Inbound Anomaly Score Exceeded (Total Score: 8)"] [ver "OWASP_CRS/4.22.0-dev"] [tag "anomaly-evaluation"] [tag "OWASP_CRS"] [hostname "192.168.56.104"] [uri "/vulnerabilities/sqli/"] [unique_id "aXmaNiwJUvbRokpxhgULRgAAAAE"]
Apache-Error: [file "apache2_util.c"] [line 286] [level 3] ModSecurity: Warning. Unconditional match in SecAction. [file "/etc/modsecurity/coreruleset/rules/RESPONSE-980-CORRELATION.conf"] [line "98"] [id "980170"] [msg "Anomaly Scores: (Inbound Scores: blocking=8, detection=8, per_pl=8-0-0-0, threshold=5) - (Outbound Scores: blocking=0, detection=0, per_pl=0-0-0-0, threshold=4) - (SQLI=5, XSS=0, RFI=0, LFI=0, RCE=0, PHPI=0, HTTP=0, SESS=0, COMBINED_SCORE=8)"] [ver "OWASP_CRS/4.22.0-dev"] [tag "reporting"] [tag "OWASP_CRS"] [hostname "192.168.56.104"] [uri "/vulnerabilities/sqli/"] [unique_id "aXmaNiwJUvbRokpxhgULRgAAAAE"]
```

7. Conclusion

This project demonstrates the practical application of **Secure Software Design** principles by protecting a vulnerable web application through secure deployment architecture. By integrating ModSecurity and OWASP CRS, the system effectively detects and blocks web-based attacks, highlighting the importance of defense-in-depth and secure configuration in modern software systems.

8. References

[1] OWASP Foundation, *OWASP Top 10 Web Application Security Risks*, OWASP, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. **Accessed: Jan. 2026**.

[2] OWASP Foundation, *OWASP ModSecurity Core Rule Set (CRS)*, OWASP. [Online]. Available: https://coreruleset.org/. **Accessed: Jan. 2026**.

[3] Apache Software Foundation, *Apache HTTP Server Documentation*. [Online]. Available: https://httpd.apache.org/docs/. **Accessed: Jan. 2026**.

[4] Docker Inc., *Docker Documentation*. [Online]. Available: https://docs.docker.com/. **Accessed: Jan. 2026**.

[5] National Institute of Standards and Technology (NIST), *Secure Software Development Framework (SSDF)*, NIST Special Publication. [Online]. Available: https://csrc.nist.gov/Projects/ssdf. **Accessed: Jan. 2026**.